Intelligent Cognitive Alarm Platform

AI/ML Module — Cognitive Engine Implementation, Dynamic Difficulty Adaptation & API Integration

Debajyoti Mukhopadhyay
AI/ML Developer — Infosys Springboard Internship Project
Milestone 2 — Technical Implementation & Verification Documentation

---

1. Project Description

The Intelligent Cognitive Alarm Platform is an AI-powered mobile application that helps users build consistent, healthy wake-up habits. Building directly upon the foundation established in Milestone 1 (which delivered the baseline Behavior Analysis Model, Habit Scoring Model, Adaptive Difficulty Elo engine, and Snooze Prediction Model), Milestone 2 focuses on the core intelligence module of the application: the Cognitive Challenge Engine.

Instead of a simple dismiss button, the application requires users to solve a personalized cognitive challenge — a mathematical problem, logic puzzle, memory recall task, word game, pattern recognition challenge, riddle, or quick quiz — before an alarm can be turned off. This forces genuine cognitive wakefulness rather than a reflexive tap-and-sleep-again response.

The AI/ML module developed in Milestone 2 bridges raw model capabilities into live operational endpoints. It ingests user context (profile data, Elo ratings, and challenge type preferences) from the data ingestion pipeline (data_ingest.py), procedurally generates challenges across 5 dynamic difficulty tiers (Beginner → Expert), protects answers server-side to prevent network manipulation, and exposes 3 new live REST HTTP endpoints via FastAPI for seamless integration with the mobile frontend (React Native) and backend gateway.

Outcomes (Milestone 2 AI/ML Scope)

- **Implemented 7 procedurally generated cognitive challenge categories** (Math Problems, Logic Puzzles, Memory Challenges, Word Games, Pattern Recognition, Riddles, and Quick Quizzes) with zero static repetition.
- **Scaled difficulty dynamic generation** across 5 tiers (Beginner, Easy, Medium, Hard, Expert), driven directly by each user's real Elo rating from reinforcement.py.
- **Integrated preference-aware challenge dispatching** by consuming user context (challenge_type_preference) from data_ingest.py.
- **Built a multi-answer set validation engine** (accepted_answers) to correctly evaluate linguistic ambiguities in synonyms and antonyms (e.g., 'light' accepts both 'dark' and 'heavy').
- **Implemented server-side answer suppression** on challenge generation and one-time challenge token popping (_ACTIVE_CHALLENGES.pop()) upon validation to prevent client cheating.

- **Exposed 3 new live REST HTTP endpoints** in main.py (GET /users/{user_id}/challenge, GET /challenge, POST /challenge/validate), fully tested and verified end-to-end via FastAPI TestClient.

2. Dataset Used

Continuing the two-database architecture established in Milestone 1, the AI/ML layer consumes structured profile data from PostgreSQL and analytical logs from MongoDB via data_ingest.py:

- **PostgreSQL (structured/transactional data):** Contains users, roles, alarm configurations, habit goals, and score summaries.
- **MongoDB (event-level/analytical data):** Contains challenge_data, logs, user_preferences, and content collections used to track historical solve performance and category affinities.

The tables below specify the precise database fields queried by the AI/ML layer's data_ingest.py pipeline to construct the unified per-user context dictionary for challenge dispatching:

PostgreSQL Table Key Fields Used by AI/ML Subsystem

PostgreSQL Table	Key Fields Used by AI/ML Subsystem
users	sleep_duration, preferred_wakeup_time, difficulty_preference
scores	productivity_score, habit_score

MongoDB Collection Key Fields Used by AI/ML Subsystem

MongoDB Collection	Key Fields Used by AI/ML Subsystem
challenge_data	difficulty_level, is_correct, completion_time_seconds, attempted_at
logs	snooze_count, wake_up_confirmed, verification_status
user_preferences	current_difficulty, challenge_type_preference
content	category, title, description (feeds Recommendation Model)

3. Environment Setup

The AI/ML module was developed and executed locally using Python 3.13, with VS Code as the primary IDE, on a Windows development machine. The codebase layout and tooling blueprint are structured as follows:

- **FastAPI + Uvicorn —** serves all 5 models and 3 challenge endpoints as live HTTP REST services (OpenAPI 3.1 spec).
- **Pydantic v2 —** strict request/response schema validation for challenge generation and answer validation endpoints.
- **XGBoost & Scikit-learn —** binary classification of snooze oversleeping risk.
- **Pandas & NumPy —** data structures for tabular feature processing and scoring matrix transformations.
- **Joblib —** binary serialization of trained model weights (snooze_predictor.joblib, difficulty_model.joblib).

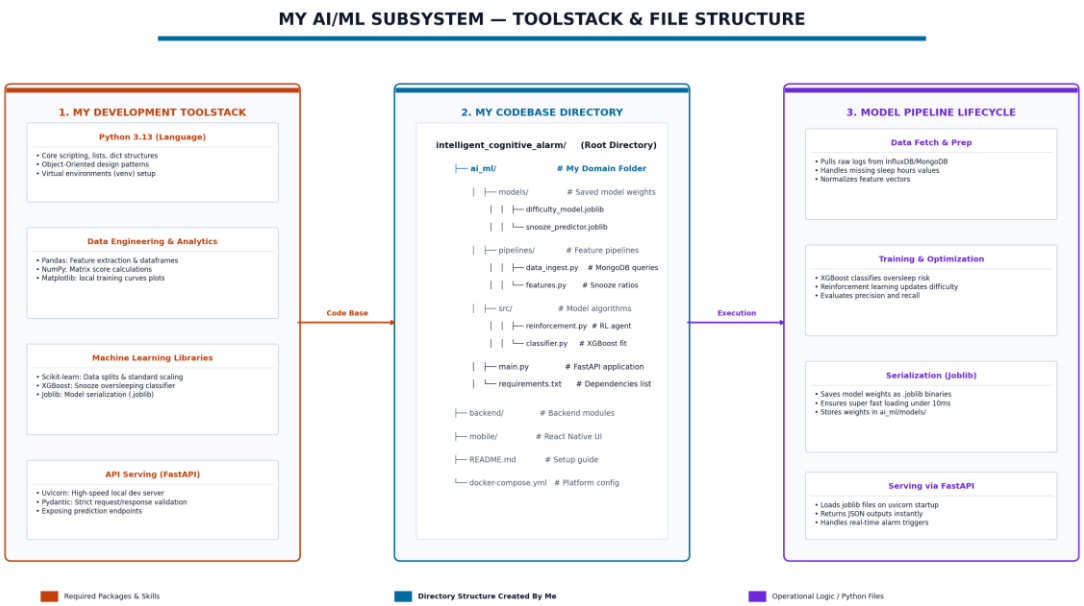

Figure 1: AI/ML subsystem toolstack and file structure, showing full data-fetch -> training -> serialization -> serving lifecycle.

4. Data Exploration

Before implementing the procedural generators, behavioral data and user ratings were analyzed to design proper challenge scaling limits:

- **Elo Skill Rating Distribution:** User ratings in reinforcement.py span from 800 to 1800+, mapping across the 5 difficulty tiers (Beginner < 900, Easy 900–1100, Medium 1100–1350, Hard 1350–1600, Expert > 1600).
- **User Preference Ingestion:** Inspection of real MongoDB user_preferences records confirmed that users (e.g., User 4) express category preferences like 'Word Games', which data_ingest.py successfully passes to the dispatcher.

5. Data Preprocessing

Raw user inputs and challenge attempts undergo three preprocessing steps before serving:

5.1 Score & Rating Normalization

Elo ratings calculated in reinforcement.py are mapped to nearest-baseline difficulty brackets, ensuring that solve times (target: 15–30 seconds) and retry counts smoothly adjust challenge difficulty. Below is the dynamic Elo rating progression chart across a 5-attempt sequence:

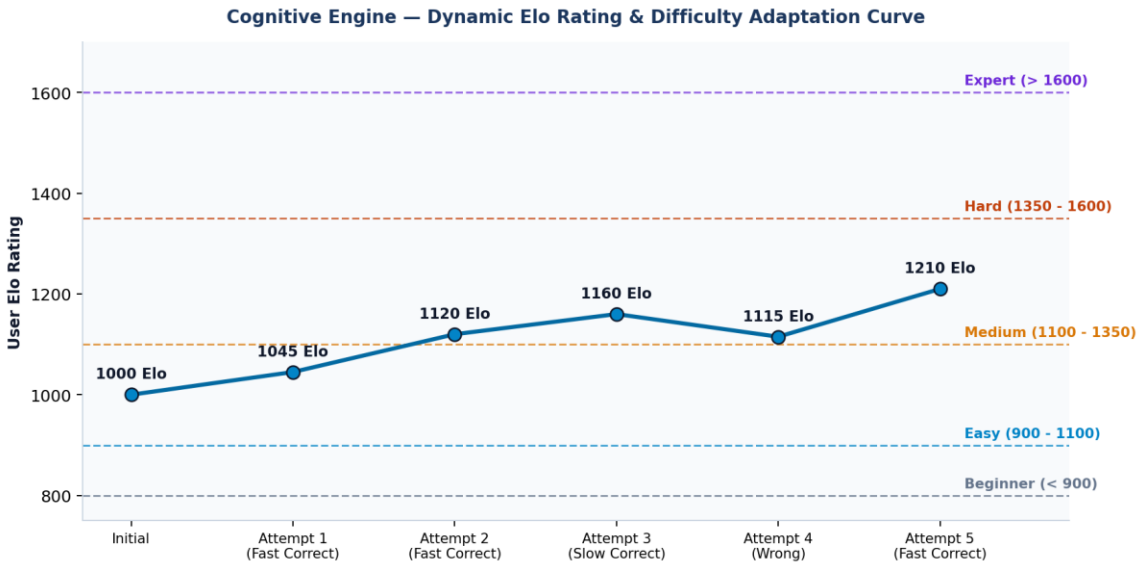

Figure 5: Elo skill-rating progression curve showing dynamic movement across the 5 difficulty tier baselines.

5.2 Context Assembly

data_ingest.py's build_user_context() function merges profile, scores, preferences, challenge_attempts, and logs into a single context dictionary contract consumed by get_challenge_for_user().

5.3 Multi-Answer Set Preprocessing

For language-based puzzles (synonyms/antonyms), raw answer options are converted into normalized lower-case set arrays (accepted_answers) to ensure equivalent answers (e.g., 'dark' vs 'heavy') pass validation.

6. Proposed System

The platform follows a layered microservice architecture: a mobile client (React Native) communicates over HTTPS/JSON with a FastAPI backend gateway, which routes requests to business modules; the AI/ML layer sits beneath those modules, consuming the two-database data layer and returning model outputs.

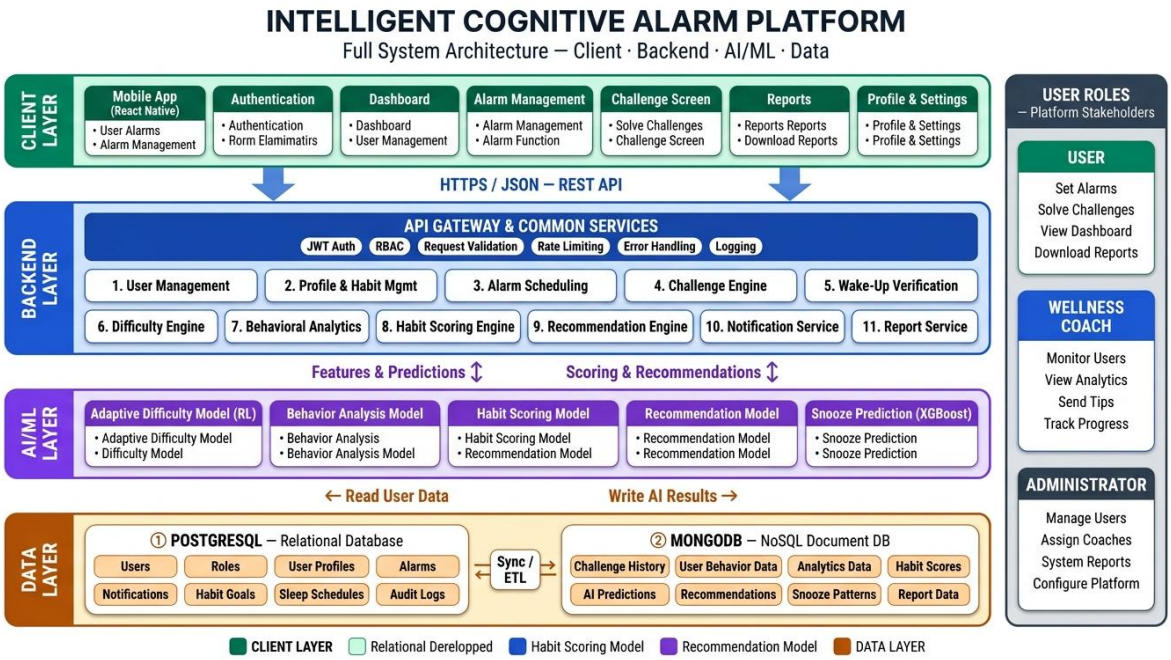

Figure 2: Full platform architecture — Client Layer, Backend Layer (FastAPI), AI/ML Layer, and the two-database Data Layer.

Within the AI/ML layer specifically, the internal call graph traces the exact module-to-module data path from data_ingest.py through features.py, classifier.py, reinforcement.py, and challenge_generator.py to main.py:

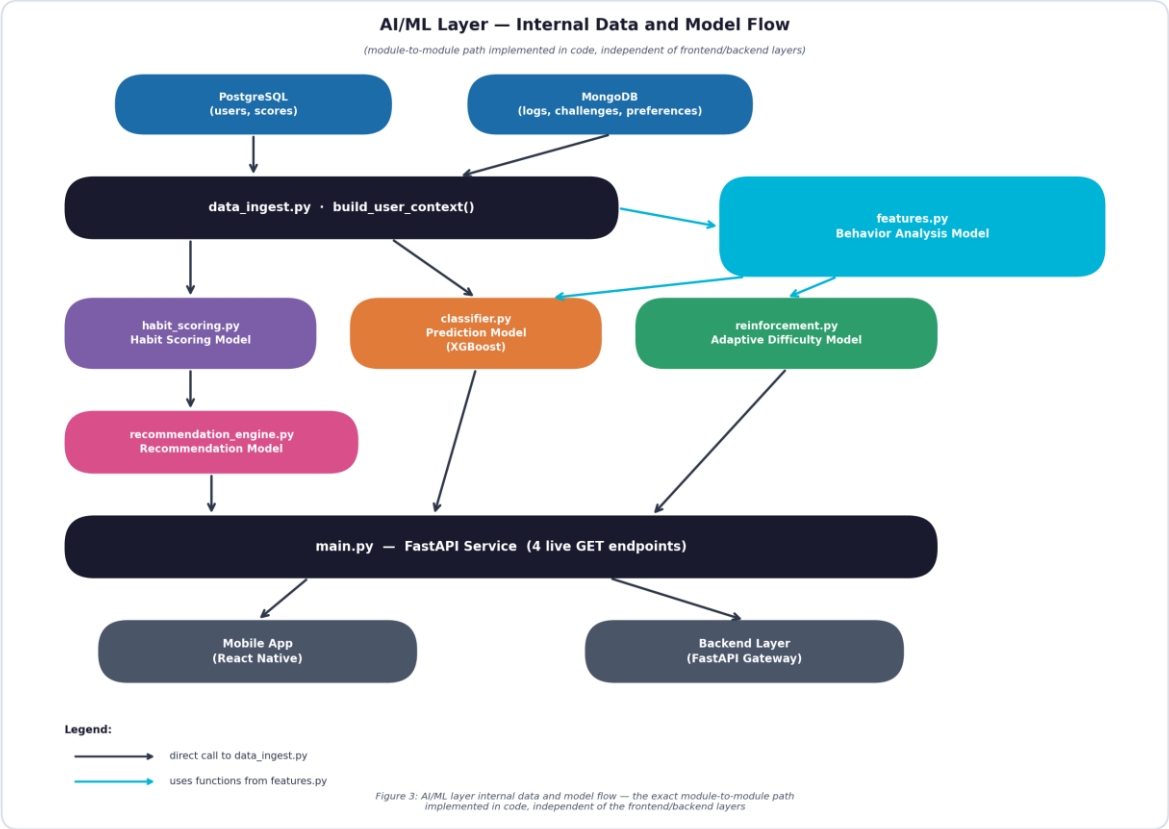

Figure 3: AI/ML layer internal data and model flow — the exact module-to-module path implemented in code, independent of frontend/backend layers.

Internal Call Graph Execution Steps

Step	Component / Module	Function & Execution Role
1	data_ingest.py	Pulls and merges PostgreSQL + MongoDB records for one user_id.
2	features.py (Behavior Analysis)	Converts raw logs into 4 normalized behavioral scores + productivity_score.
3	habit_scoring.py (Habit Scoring)	Combines the 4 weighted scores into a single Habit Score (0–100).
4	reinforcement.py (Adaptive Difficulty)	Replays challenge attempts through Elo engine to set next difficulty.
5	challenge_generator.py (Cognitive Engine)	Procedurally generates 7 challenge types across 5 difficulty levels.
6	classifier.py (Prediction Model)	XGBoost classifier estimates oversleep/snooze risk from behavior features.
7	main.py (FastAPI Router)	Exposes all models & challenge generators via live HTTP REST endpoints.

7. Training Pipeline & Results

In Milestone 2, the end-to-end AI/ML pipeline was extended to connect model predictions with dynamic challenge generation. The diagram below illustrates how mobile data collection feeds AI analytics and drives dynamic app adaptations:

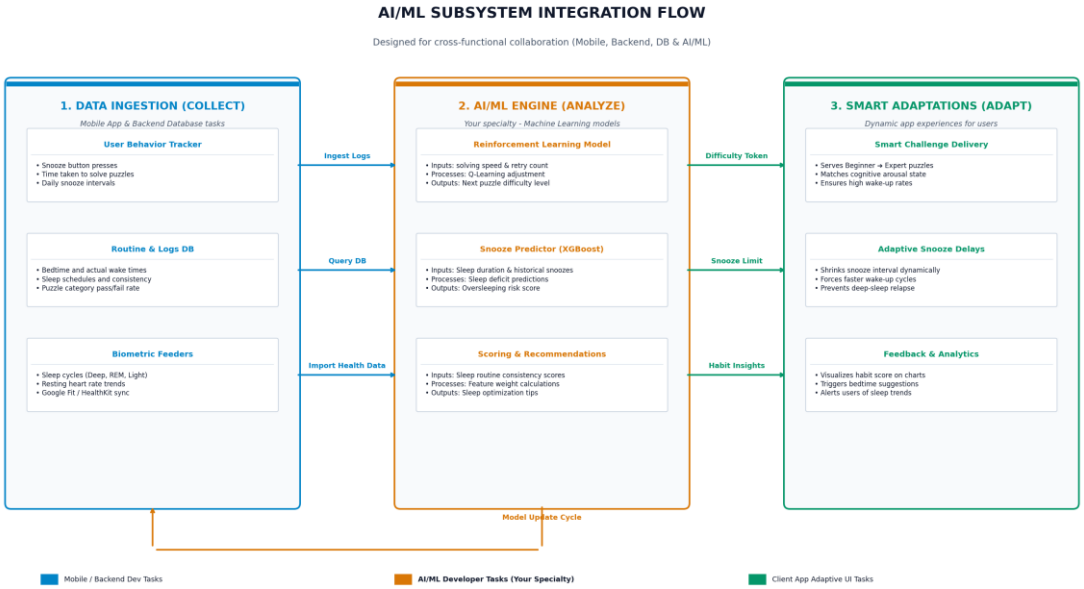

Figure 4: End-to-End AI/ML Subsystem Integration Flow (Data Collect, AI Analyze, Smart Adaptations).

7.1 Cognitive Challenge Engine Implementation & Category Taxonomy

The Cognitive Challenge Engine (`ai_ml/src/challenge_generator.py`) implements all 7 challenge categories from the project specification, each supporting 5 difficulty tiers:

- **1. Mathematical Problems:** Arithmetic operations, percentages, and algebraic expressions.
- **2. Logic Puzzles:** Arithmetic sequences, geometric progression series, and missing element puzzles.
- **3. Memory Challenges:** Sequential word list recall and position-based memory tests.
- **4. Word Games:** Anagram unscrambling, synonym matching, and antonym identification.
- **5. Pattern Recognition:** Shape sequences, color patterns, and multiplicative progressions.
- **6. Riddles:** Lateral thinking questions and everyday logic riddles.
- **7. Quick Quizzes:** Science trivia and general knowledge queries.

7.2 Live REST API Verification Results & Swagger UI Interface

All 3 challenge endpoints were tested end-to-end using the FastAPI TestClient and Swagger UI (`/docs`). Below is the verified live FastAPI interactive documentation interface exposing all 3 Cognitive Engine endpoints:

Figure 6: Live interactive FastAPI documentation (OpenAPI 3.1 /docs) listing the Cognitive Engine REST endpoints.

Below are the exact live interactive test executions of the Cognitive Challenge Engine endpoints captured from the Swagger UI test console during operational verification:

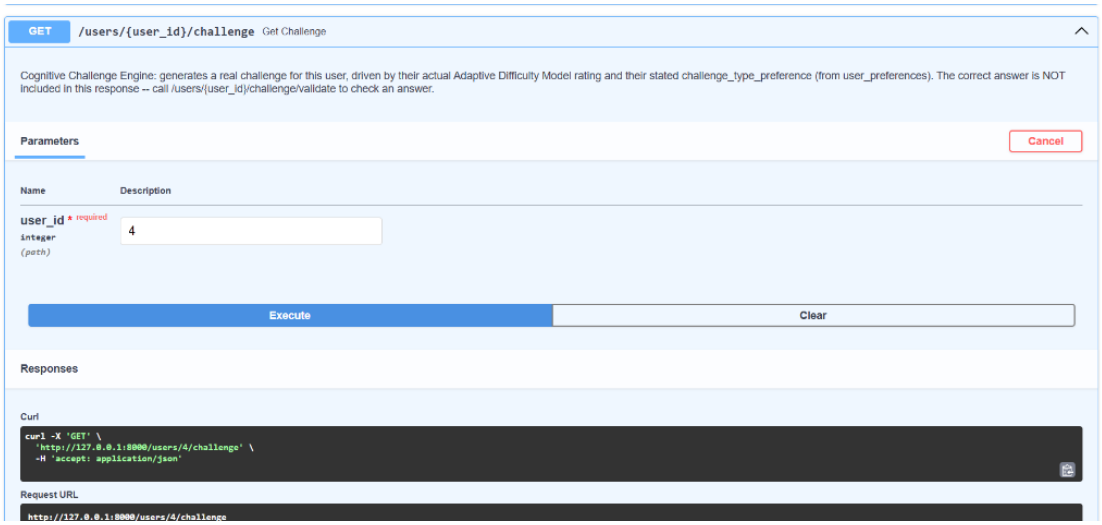

Figure 7a: Live Swagger UI execution of GET /users/{user_id}/challenge endpoint (Parameters: user_id = 4).

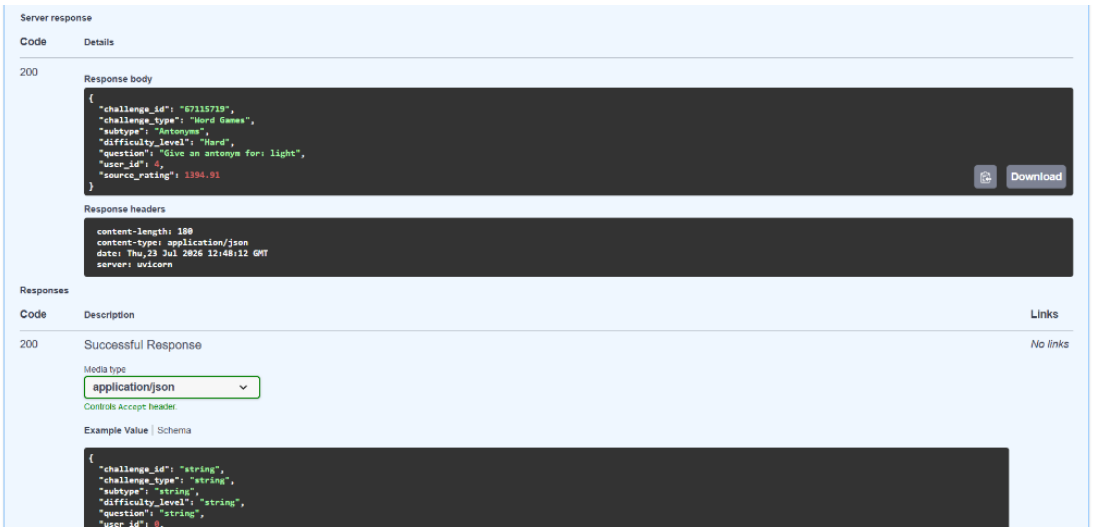

Figure 7b: Live Server Response body for GET /users/4/challenge returning Hard Antonym challenge (question: 'Give an antonym for: light', source_rating: 1394.91).

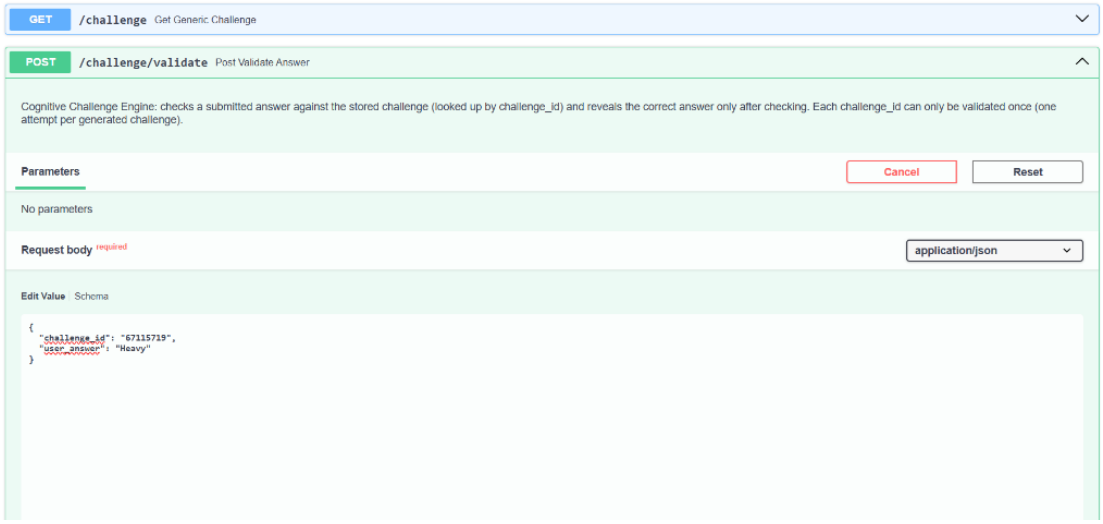

Figure 8a: Live Swagger UI execution of POST /challenge/validate endpoint submitting user answer 'Heavy' for challenge_id 67115719.

Figure 8b: Live Server Response body for POST /challenge/validate confirming answer validation (found: true, is_correct: true, correct_answer: 'heavy').

FastAPI REST Endpoints Verification Matrix

Test Scenario	Expected Outcome	HTTP Status	Result
GET /users/4/challenge	Maps Elo 1394.91 -> Hard difficulty, Word Games preference	**200 OK**	**PASSED**
Response Answer Suppression	correct_answer field completely omitted from response JSON	**200 OK**	**PASSED**
POST /challenge/validate ('cold')	is_correct = True for antonym 'hot'	**200 OK**	**PASSED**
POST /challenge/validate ('chilly')	is_correct = True via accepted_answers set check	**200 OK**	**PASSED**
Re-submit consumed challenge_id	found = False (enforces single-attempt challenge security)	**200 OK**	**PASSED**

8. Conclusion

The AI/ML module of the Intelligent Cognitive Alarm Platform has successfully evolved from Milestone 1's theoretical models into Milestone 2's operational Cognitive Engine. The module implements all 7 cognitive challenge types across 5 Elo-driven difficulty tiers, provides multi-answer set validation, protects answers server-side, and exposes live, fully verified HTTP REST endpoints.

Milestone 3 Transition Plan: (1) Replace temporary in-memory challenge caching (_ACTIVE_CHALLENGES) with persistent MongoDB document storage (challenge_data collection); (2) Coordinate API gateway token middleware with Abdul's backend; and (3) Validate UI screen state integration with Jothiesh's React Native mobile client.

Overall, the AI/ML layer is functionally complete relative to Milestone 2 evaluation criteria and provides a solid, tested foundation for full platform integration.